token. To ensure that users have authenticated using the `UserController` before they hit the `TokenController` login endpoint, you should add the new endpoint after the existing authentication filters. Given that logging in is an important action from a security point of view, you should also make sure that calls to the login endpoint are logged by the `AuditController` as for other endpoints. To add the new login endpoint, open the Main.java file in your editor and add lines to create a new `TokenController` and expose it as a new endpoint, as in listing 4.6. As you don't yet have a real `TokenStore` implementation, you can pass a `null` value to the `TokenController` for now. Rather than have a `/login` endpoint, we'll treat session tokens as a resource and treat logging in as creating a new session resource. Therefore, you should register the `TokenController` login method as the handler for a POST request to a new `/sessions` endpoint. Later, you will implement logout as a DELETE request to the same endpoint.

Listing 4.6 The login endpoint

```
TokenStore tokenStore = null;     #A
var tokenController = new TokenController(tokenStore);    #A

before(userController::authenticate);    #B

var auditController = new AuditController(database);    #C
before(auditController::auditRequestStart);    #C
afterAfter(auditController::auditRequestEnd);    #C


before("/sessions",
userController::requireAuthentication);    #D
```